
Titulo del TFG
Title in english



Trabajo de Fin de Grado
Curso 2025–2026

Autor
Eva Paula Mik

Director
Ismael Sagredo Olivenza

Colaborador
Colaborador no definido. Usa \colaboradorPortada

Grado en Desarrollo de Videojuegos
Facultad de Informática
Universidad Complutense de Madrid

Titulo del TFG
Title in english

Trabajo de Fin de Grado en Desarrollo de Videojuegos

Autor
Eva Paula Mik

Director
Ismael Sagredo Olivenza

Colaborador
Colaborador no definido. Usa \colaboradorPortada

Convocatoria: *Junio 2026*

Grado en Desarrollo de Videojuegos
Facultad de Informática
Universidad Complutense de Madrid

30 de abril de 2026

Dedicatoria

*A toda la gente que nos ha hecho llegar hasta
aquí,
un besazo.
A todos los que han estado a nuestro lado.*

Agradecimientos

Os amo!

Mik

Os amo!

Pau

Os amo!

Eva

Resumen

Titulo del TFG

Nuestro estudio se basa en la comparativa de distintos modelos de IA para la frente a la simulación de físicas costosas en proyectos 3D en motores de videojuegos. Para ello creamos una simulación en Unity con un objeto con componente Cloth y distintos colliders que interactuasen con ella. Tras generar los datasets, probamos distintos modelos de IA e integramos los datos. Finalmente exportamos a unity con Sentis y realizamos comparativas teniendo en cuenta GPU, CPU. Observamos que el modelo que mejores resultados obtuvo en cuanto a tal métrica fue X.

Palabras clave

Inteligencia Artificial, Simulación física, Unity, Videojuegos, Optimización, PyTorch, ML

Abstract

Title in english

We need to do this.

Keywords

We need to do this.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Plan de trabajo	2
1.4. Metodología	3
1.5. Estructura de la memoria (propuesta)	4
2. Estado de la Cuestión	5
2.1. Simulación física en videojuegos	5
2.1.1. Motores de física	6
2.1.2. Motores de videojuegos	6
2.2. Simulación de telas	7
2.2.1. Cloth	7
2.2.2. El problema de optimización	8
2.2.3. Soluciones	8
2.3. Inteligencia Artificial en videojuegos	9
2.3.1. Librerías	9
2.3.1.1. Pytorch	9
2.3.1.2. ONNX	10
2.3.1.3. Sentis	10
2.3.2. Modelos de optimización de telas	10
2.3.2.1. Towards Generalized Position-Based Dynamics	10
2.3.2.2. Neural Cloth Simulation	11
2.3.2.3. Subspace Neural Physics: Fast Data-Driven Interactive Simulation	11
2.3.2.4. Cloth and Skin Deformation with a Triangle Mesh Based Convolutional Neural Network	12
2.3.3. DeepSD: Real-time Cloth Simulation with Deep Learning	13
2.3.3.1. PBNS: Physically Based Neural Simulation for Un-supervised Garment Pose Space Deformation	13
2.3.3.2. SNUG	13

2.3.3.3. HOOD: Hierarchical Graphs for Generalized Modeling of Clothing Dynamics	14
3. Descripción del Trabajo	15
3.1. SimulacionUnity	15
3.2. Dataset	15
3.3. EntrenamientoPython	15
3.4. SimulacionUnity	15
3.5. ModelosFuncionales	15
3.6. PruebaTesters	15
4. Conclusiones y Trabajo Futuro	17
4.1. Conclusiones	17
4.1.1. Conclusiones de la búsqueda de datasets	17
4.1.2. Conclusiones de la blablabla	17
4.1.3. Limitaciones	17
4.2. Trabajo Futuro	17
Contribuciones Personales	19
Bibliografía	21
A. Carteles para la generación del dataset	25
B. Formulario de recogida de citas	29
C. Resultados de los distintos modelos CNN	31
C.1. Intervalo 0.2s	31
C.2. Intervalo 0.4s	33
C.3. Intervalo 0.6s	34
C.4. Intervalo 0.8s	36
C.5. Intervalo 1.0s	37
Glosario	39
Licencia de uso del documento	41
Licencia de uso del código fuente	43

Índice de figuras

1.1. Diagrama de Gantt del proyecto	3
A.1. Cartel colgado en la facultad de informática	25
A.2. Cartel colgado en redes sociales	26
A.3. Cartel colgado en pantallas de la facultad	27
B.1. Formulario de recogida de citas (primera parte)	29
B.2. Formulario de recogida de citas (segunda parte)	30
C.1. Esquema del modelo CNN con 0.2s de intervalo	31
C.2. Matriz de confusión del modelo CNN con 0.2s de intervalo	32
C.3. Gráfico de entrenamiento del modelo CNN con 0.2s de intervalo (mejor val_accuracy = 0.28777)	32
C.4. Esquema del modelo CNN con 0.4s de intervalo	33
C.5. Matriz de confusión del modelo CNN con 0.4s de intervalo	33
C.6. Gráfico de entrenamiento del modelo CNN con 0.4s de intervalo (mejor val_accuracy = 0.49473)	34
C.7. Esquema del modelo CNN con 0.6s de intervalo	34
C.8. Matriz de confusión del modelo CNN con 0.6s de intervalo	35
C.9. Gráfico de entrenamiento del modelo CNN con 0.6s de intervalo (mejor val_accuracy = 0.44952)	35
C.10. Esquema del modelo CNN con 0.8s de intervalo	36
C.11. Matriz de confusión del modelo CNN con 0.8s de intervalo	36
C.12. Gráfico de entrenamiento del modelo CNN con 0.8s de intervalo (mejor val_accuracy = 0.40583)	37
C.13. Esquema del modelo CNN con 1.0s de intervalo	37
C.14. Matriz de confusión del modelo CNN con 1.0s de intervalo	38
C.15. Gráfico de entrenamiento del modelo CNN con 1.0s de intervalo (mejor val_accuracy = 0.56002)	38

Índice de tablas

Introducción

En los últimos años, el aumento en el tamaño y la complejidad en los videojuegos ha supuesto la imprescindible optimización y automatización de diversos aspectos de su desarrollo, tanto en el ámbito del renderizado como en la lógica interna de los sistemas. Particularmente, en la evolución de entornos 3D con el objetivo de obtener gráficos más detallados, lo que conlleva un incremento significativo en el coste computacional.

Elementos como el cabello, los fluidos o las telas requieren una carga de cálculos que dificulta el alcance de comportamientos realistas en tiempo real, lo que afecta negativamente al rendimiento de los videojuegos, en especial en términos de tasa de fotogramas. En consecuencia, se ha producido un avance en la recogida de técnicas basadas en ML y DL en la industria de los videojuegos.

Empresas pioneras en hardware y software han desarrollado tecnologías para mejorar el rendimiento gráfico incorporando esta clase de algoritmos. Entre ellas destacan: NVIDIA con DLSS (Deep Learning Super Sampling), donde tenemos el Resident Evil Requiem que usa DLSS 4; y AMD con FSR (FidelityFX Super Resolution). Asimismo, varias compañías lo incorporan en el propio curso del juego simulando comportamientos y mecánicas como EA (FIFA), Embark Studios (Arc Raiders) o Ubisoft con sus múltiples líneas de investigación con la IA.

En este contexto, la aplicación de modelos de aprendizaje automático en simulaciones físicas se presenta de forma prometedora.

1.1. Motivación

Las simulaciones físicas no solo se dan en sistemas de alto rendimiento con hardware especializado y gráficas delicadas, sino que también deben ejecutarse en máquinas con recursos más limitados. Esta necesidad surge por el crecimiento de distintas plataformas, incluyendo a su vez otros tipos de dispositivos, como los móviles.

En motores de videojuegos como Unity, más utilizado por desarrolladores indie, se utiliza principalmente la CPU para calcular físicas, lo que puede convertirse en un cuello de botella cuando se pasa a escenarios más complejos. Esto conllevaría, por ejemplo, a simplificar modelos, reducir la calidad de las animaciones o limitar la

interacción física, afectando al desarrollo del producto y la experiencia de usuario. Creemos que incorporar la IA puede ser clave para este reto técnico, permitiendo agilizar el proceso en máquinas de menor rendimiento.

Frente a esta problemática, el uso de modelos de aprendizaje automático ofrece una alternativa para aproximar un comportamiento físico detallado con un menor coste computacional en tiempo de ejecución. Este enfoque implica una carga elevada al inicio de entrenamiento de modelos, compensado con una ejecución eficiente en los próximos pasos. Cabe destacar que, si bien utilizar la IA es sacrificar que las físicas sean precisas, creemos que esto es caer en la falacia de la inmersión [Salen y Zimmerman \(2004\)](#). Nuestro objetivo es lograr simulaciones visualmente verosímiles que mantengan una percepción fiel de realismo, adoptando así una perspectiva cercana al diseño de videojuegos. Por ello, se ha escogido Cloth, simulación de telas, como caso de estudio debido a que representa un equilibrio adecuado entre accesibilidad tecnológica, complejidad física e impacto visual. De esta manera, realizaremos un análisis tanto técnico como perceptivo.

1.2. Objetivos

El objetivo general de este proyecto es la implementación de un modelo de IA que permita replicar el movimiento en tiempo real de una tela en un motor de videojuegos con baja latencia. Para cumplir con este objetivo, se plantean las siguientes metas durante el desarrollo del proyecto:

1. Generar una animación de una tela y un colisionador que varíe su trayectoria en cada ejecución de la escena.
2. Generar y procesar datasets con la información relevante de la tela respecto a su entorno y colisionador.
3. Implementar, entrenar y evaluar los distintos modelos de aprendizaje automático, tales como redes neuronales (LSTM, CNN y RNN, GNN).
4. Analizar y comparar los resultados visuales de los distintos modelos.

1.3. Plan de trabajo

El desarrollo del proyecto se llevará a cabo en el periodo lectivo del curso 2025-2026. Dados los objetivos del apartado anterior, la planificación se divide en las siguientes iteraciones:

- **Iteración 0:** En esta etapa se concretan los objetivos reales de la investigación y se establece la planificación del trabajo respecto a la propuesta inicial.
- **Iteración 1:** Etapa de preparación, donde se cumplirán los siguientes subobjetivos para implementar un sistema de simulación adecuado para ser procesado por los modelos en un futuro.

- **Iteración 1.1:** Se realizará una investigación de trabajos relacionados con nuestro tópico, relevante para dirigir el desarrollo de nuestros experimentos. Se analizarán distintas alternativas y se decidirá cuales resultarán más interesantes de replicar, se definirán qué tipo de datos deberemos recopilar para el entrenamiento y qué tipo de redes podrán llevarlo a cabo.
- **Iteración 1.2:** Se creará una simulación básica en Unity y un sistema de registro de datos relevantes a la tela.
- **Iteración 2:** Se establecerá un entorno de trabajo para el entrenamiento de los datos recopilados. Se generará un pipeline claro en el que se integre la simulación de recogida de datos, su limpieza y posterior uso para entrenar el modelo y, finalmente, la incorporación de este al entorno de simulación. Para poder asegurarnos del correcto funcionamiento de este sistema se generará un modelo simple y se comprobará que el pipeline es correcto. En esta fase se comenzarán a redactar los primeros capítulos de la memoria.
- **Iteración 3:** Con una base sólida ya montada, se aumentará la complejidad y, por consiguiente, el coste de la simulación. Se implementarán distintos modelos y se llevarán a cabo pruebas con todos ellos para asegurarse de su fiabilidad.
- **Iteración 4:** Si las anteriores iteraciones funcionan de manera correcta esta última no tendrá ninguna carga de implementación. Se alcanzará la última meta definida: hacer distintas pruebas entrenando los modelos ya desarrollados, sacar métricas de todos ellos para poder hacer comparaciones y hacer un análisis exhaustivo de estos resultados, sacando así las conclusiones de la investigación.

El desarrollo de las tareas y su planificación se puede ver en la figura 1.1.

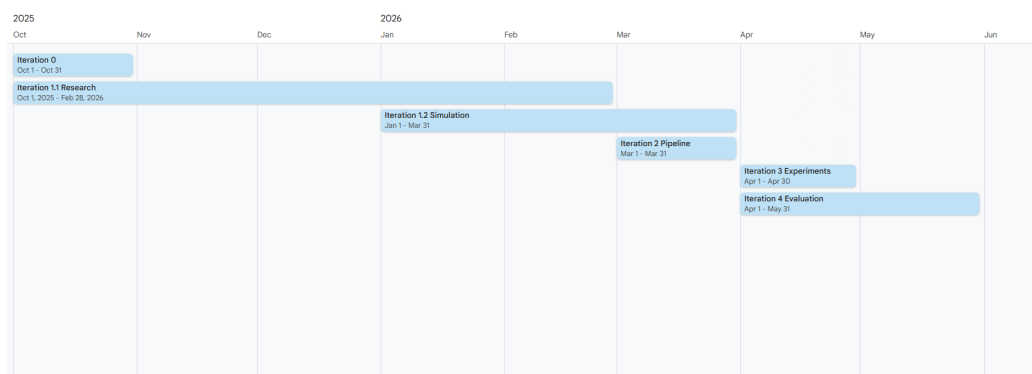


Figura 1.1: Diagrama de Gantt del proyecto

1.4. Metodología

Para el desarrollo de esta investigación se emplearán herramientas de libre acceso y uso gratuito, con el objetivo de facilitar la reproducibilidad de los resultados obtenidos.

El entorno de desarrollo principal será Windows 11, al tratarse del sistema operativo utilizado por el equipo de trabajo y extendido entre los potenciales usuarios. La simulación de la tela, así como la integración del modelo de inteligencia artificial, se llevarán a cabo en Unity (versión 6000.3.2f1, correspondiente a una versión de soporte a largo plazo). En cuanto a los lenguajes de programación, se empleará C# para el desarrollo dentro de Unity, incluyendo la generación de la simulación y la recopilación de datos, mientras que se utilizará Python, empleando Pytorch,v para el entrenamiento de los modelos de aprendizaje automático. La integración de los modelos entrenados en el entorno de ejecución se realizará a través de Unity Sentis.

Para la gestión del código fuente y el trabajo colaborativo se utilizará GitHub, basado en el sistema de control de versiones Git. Asimismo, se empleará la herramienta GitHub Projects para la planificación, asignación y seguimiento de tareas. Como herramientas adicionales, se utilizará Google Drive para el almacenamiento de documentación, apuntes de reuniones y materiales de trabajo puntuales, y Discord para la coordinación del equipo mediante reuniones online, además de encuentros presenciales semanales.

1.5. Estructura de la memoria (propuesta)

Capítulo 2

Estado de la Cuestión

En este capítulo se presenta una breve descripción del desarrollo de las simulaciones físicas en videojuegos, así como el hardware y software que ha hecho que esta evolución sea posible, se presenta la simulación de telas en específico, las maneras que existen de implementarla, los problemas de optimización que estas suponen y los avances en modelos de IA que permiten imaginar otras soluciones.

2.1. Simulación física en videojuegos

Basta con remontarse a la creación del Pong en 1972, para darse cuenta de la importancia que las simulaciones físicas han tenido en el ámbito de los videojuegos desde el primer momento. Las físicas permiten crear situaciones del gameplay que no están predefinidas, enriqueciendo inmensamente la experiencia del jugador.

En los inicios del desarrollo de videojuegos, las técnicas más rudimentarias para implementar estas físicas incluían:

- **La prerenderización:** Dada la capacidad de cómputo con la que solían contar las máquinas, era imposible hacer todos los cálculos físicos en tiempo real, por lo que las interacciones físicas podían diseñarse previamente para parecer realistas [Templet \(2021\)](#).
- **La implementación forzada de cada elemento:** El comportamiento de cada elemento podía ser implementado únicamente para él, en vez de seguir unas normas físicas generales. Por ejemplo, en el caso de un proyectil, podía codificarse calculando su trayectoria y velocidad en pocas líneas de código.

Estos métodos eran claramente limitados en la precisión que ofrecían, y quedaba clara la necesidad de una solución más general y reproducible. El mismo Ian Millington nos introduce en su libro *Game Physics Engine Development* [Millington \(2007\)](#) el juego Exile, creado por Peter Irvin y Jeremy Smith. Si bien este juego fue publicado en 1988, ya contaba con conservación del momento en colisiones, gravedad, y fuerzas como el viento o explosiones. Lo resaltamos como uno de los primeros

juegos que cuenta con un motor de físicas completo.

2.1.1. Motores de física

La creación de motores de física completos aportó a los videojuegos escalabilidad en su gameplay, manteniendo la coherencia de sus elementos físicos. Esto último resulta imprescindible para la inmersión del jugador [Hecker \(2000\)](#).

En los años 90, con la popularidad del 3D y gráficos cada vez más avanzados, la física era el próximo área en la que marcar la diferencia: la evolución de estos motores era inevitable. Los grandes cálculos en tiempo real con su correspondiente carga para la CPU plantearon dos grandes retos: el avance de los motores físicos cada vez más precisos y potentes, y, por otro lado, la optimización del hardware para llegar a la demanda computacional que esto suponía.

En 2002, se funda Ageia Technologies, responsable de la creación del primer procesador de física dedicado para ordenadores, denominado PhysX PPU (Physics Processing Unit), así como del motor PhysX [NVIDIA \(2026\)](#), que sigue siendo líder en la industria a día de hoy. Posteriormente, NVIDIA Corporation, conocida por su papel en la producción de GPUs, adquiere Ageia, impulsando el cálculo de físicas en GPU. Esto sumado a la dificultad de penetrar en el mercado del hardware hace que la producción de PPUs quede obsoleta.

Por otro lado, Intel adquiere en 2007 Havok [Havok \(2026\)](#), otro de los primeros motores de físicas, cancelando el programa HavokFX que pretendía llevar el cálculo de físicas a la GPU para realizar estos en sus CPUs multinúcleo. Posteriormente es adquirido por Microsoft en 2015 y se integra en las herramientas de desarrollo como DirectX. Se establece como el middleware más común en grandes producciones y simulaciones complejas en estudios, mientras que Physx se convierte en open source en 2018 y se integra en motores de videojuegos como Unreal.

Los motores físicos nacen en un contexto de competición entre empresas, quienes intentan establecer su hardware como óptimo para la simulación de físicas. La disputa y diversificación para carga computacional entre CPU y GPU la veremos durante todo el desarrollo de este proyecto.

2.1.2. Motores de videojuegos

Los motores físicos, sumados a otros elementos análogos como los motores de renderizado y sonido crean el software al que nos referimos como motores de videojuegos. En el mercado encontramos establecidos motores como Unity o Unreal Engine, motores propios de grandes estudios y aquellos pequeños indies como Godot. Los motores con más presencia en el desarrollo indie son:

- **Unreal:** Es un motor de videojuegos creado por Epic Games en 1998. Destaca por motor gráfico, permitiendo gráficos realistas en tiempo real con tecnologías como Nanite para geometría virtualizada o Lumen para iluminación dinámica. Es multiplataforma y lo encontramos no sólo en desarrolladores indie sino en grandes producciones AAA. Se caracteriza por su uso de Blueprints en programación (c++). Integraba Physx hasta la introducción reciente de Chaos Physics en su versión 5.7, que aporta novedades que destacaremos en el apartado 2.2.3.
- **Unity:** Creado por Unity Technologies en 2005, abarca un porcentaje del 51 % de juegos publicados en 2024 y un 26 % [Tower \(2025\)](#) en las unidades vendidas en Steam. Es el motor más utilizado por desarrolladores indies y cuenta con documentación, gran comunidad y extensiones como Sentis para la integración de modelos de IA, que explicaremos más adelante. Unity3D integra PhysX como motor de físicas, pero puede incorporar otros sistemas de físicas como Havok bajo licencia [Technologies \(2026a\)](#). Está basado en la arquitectura Entidad-Componente en c#, y tiene un enfoque menos potente en cuanto a gráficos y tamaño del juego.

2.2. Simulación de telas

Entendemos los objetos deformables o soft-bodies como aquellos que no son rígidos. Así como en los objetos rígidos podemos simular su movimiento sabiendo su trayectoria y centro de masa, para los objetos deformables necesitamos tener en cuenta todas las partículas y propiedades físicas que lo componen. Entre estos encontramos elementos como las telas, el agua o el viento [Kenny Erleben \(2005\)](#).

El protagonista de la simulación de telas es la vestimenta: vestidos, cintas o capas moviéndose dependiendo de las acciones del jugador, viento y condiciones específicas permiten que el jugador se inmerse por completo. Además de encontrar otros objetos hechos a partir de tela como banderas, podemos incluso aplicar las físicas de la tela en otros elementos como pelo, hierba o globos; como señala Havok: “*Havok Cloth is used to create secondary motion and to enhance any object that flows*”.

En esta sección veremos cómo se representa y simula una tela en tiempo real, el problema que supone y qué tipos de soluciones encontramos. Nos centraremos mayoritariamente en ejemplos de Unity3D, como nuestro referencia de motor de videojuegos.

2.2.1. Cloth

Cloth es un componente que nos proporciona Unity3D que funciona junto al componente Skinned Mesh Renderer para simular telas. Está basado en Cloth de PhysX y fue incorporado con el objeto de poder disponer de ropa para personajes. Tiene múltiples propiedades como la resistencia a la flexión, rigidez, damping o uso

de gravedad.

Se trata la tela como una malla de partículas conectadas por restricciones. El Skinned Renderer calcula las transformaciones basándose en una malla de baja resolución sobre la que realiza los movimientos en los vértices, que dinámicamente se aplican en la malla real y obtenemos la simulación visual. Aquellos constraints no visibles o de distancia 0, fijos, no serán calculados. La tela cuenta con capacidad de colisionar internamente y con dos tipos de colliders externos: cápsulas o esferas. Cualquier objeto externo con el que se quiera colisionar tendrá que ser un conjunto de estos colliders.

Pese a encontrar múltiples acepciones en el término solver con distintas resoluciones de problemas, aquí nos referiremos como solver al algoritmo computacional que se encarga de predecir el comportamiento del objeto deformable en renderizados gráficos y simulaciones físicas, específicamente, de telas. Encontramos infinitos tipos de solvers. Los primeros solvers se basan en los algoritmos que emplean la fuerza bruta: realizan cálculos en masa para sistemas de muelles (MSS o Mass Spring Systems) [Provot \(1995\)](#). Los solvers más actuales resultan en el PBD, proceso iterativo que proyecta las posiciones de las partículas sobre un “espacio de restricciones” [Müller et al. \(2007\)](#).

El solver de PhysX se basa en PBD, es decir, trabaja directamente sobre las posiciones. Encontramos también XPBD [Macklin et al. \(2016\)](#), una versión extendida que puede incorporarse en Unity.

2.2.2. El problema de optimización

La simulación de telas en tiempo real es un problema que persiste, en el que seguimos buscando la optimización a día de hoy. El aumento de tamaño o complejidad de una tela, con su alta carga computacional, conlleva el decaimiento inmediato de frames. A parte de dificultar el realismo en juegos y demás simulaciones a tiempo real, resulta un gran impedimento en entornos donde se requieren fotogramas por segundo altos como son los juegos en realidad aumentada para evitar mareos.

Mientras que el Cloth de PhysX puede delegar la simulación a GPUs compatibles con CUDA, el Cloth integrado por defecto en Unity realiza todo su trabajo en la CPU, lo cual ralentiza mucho la ejecución. En el caso de Unreal, previamente al lanzamiento de ML Cloth Simulation, Chaos Physics delegaba su mayor fuerza de trabajo en la CPU (optimizando gracias al contacto con Intel).

2.2.3. Soluciones

Encontramos varios tipos de soluciones para la optimización de la simulación de telas:

- Optimización por CPU: encontramos la extensión de Obi Cloth [Method \(2019\)](#),

que aprovecha el sistema de Jobs de Unity para paralelizar la simulación y acelerar la simulación sin salir de la CPU. Es uno de los paquetes más populares para solucionar este problema.

- Paralelización en GPU: a través de shaders se delega la fuerza de cálculo del skinning en la GPU. Encontramos diferentes estudios que aportan diferentes enfoques para la optimización: Jim Hugunin realiza una investigación para alcanzar altas resoluciones y evitar colisiones en las mallas [Jim Hugunin \(2016\)](#), mientras que otros se centran en la mejora de rendimiento pura para adaptar simulaciones a entornos VR [Va et al. \(2021\)](#).
- Entrenamiento con IA: el enfoque más actual de la resolución de simulaciones complejas como Cloth es la Inteligencia Artificial. Se entrenan modelos de ML y DL con los datos de las mallas animadas o en simulación, con lo que obtenemos más adelante mejor rendimiento en tiempo real. Se sacrifica memoria y tiempo de entrenamiento para obtener mejores y mucho más rápidas simulaciones de tela en escena. Actualmente, nos encontramos en el paso de investigación a implementación en grandes empresas: Unreal ya proporciona una simulación de tela a través de aprendizaje automático [Games \(2025\)](#), en el que destaca su realismo. Aún así, cuenta con limitaciones al basarse un NearestNeighborSearch: él sistema es capaz de predecir detalles como arrugas pero no movimiento dinámico como ondulaciones o drapeados. En la próxima sección detallaremos las numerosas investigaciones recientes que abarcan aspectos desde la reproducción de arrugas, el compromiso entre calidad, coste y rendimiento hasta colisiones y penetraciones en las mallas.

2.3. Inteligencia Artificial en videojuegos

Como se ha introducido previamente, en la industria del videojuego, como en muchas otras, la atención está fijada en la inteligencia artificial. En este apartado presentaremos los recursos e investigaciones más relevantes para el desarrollo de este proyecto.

2.3.1. Librerías

En este subapartado describiremos las librerías que han resultado indispensables para entrenar los modelos de IA y conectarlos a la simulación en Unity.

2.3.1.1. Pytorch

Pytorch es una de las librerías de código abierto que permite escribir redes neuronales propias. Los tensores, matrices multi-dimensionales análogas a los NumPy arrays de Python, permiten optimizar las redes tanto para el uso de CPU como de GPU [Contributors \(2026\)](#). Suele ser la más elegida en entornos académicos, ya que permite prototipado rápido y entrenamiento dinámico. Fue originalmente desarrollado por Facebook, y todavía cuenta con una enorme comunidad de desarrolladores.

2.3.1.2. ONNX

ONNX es un formato creado para representar modelos de aprendizaje automático [Project \(2026\)](#). Fue introducido por Facebook y Microsoft en 2017, con el objetivo de facilitar el cambio de entornos de trabajo en aprendizaje automático.

2.3.1.3. Sentis

Sentis es una librería de inferencia de redes neuronales para Unity, predecesora introducida en el 2023 de la librería Barracuda. Permite importar modelos de redes neuronales entrenados a Unity y ejecutarlos en tiempo real tanto con la CPU como con la GPU [Technologies \(2026b\)](#).

2.3.2. Modelos de optimización de telas

La optimización de simulaciones con redes neuronales cuenta ya con un amplio recorrido, en este apartado se describen los estudios más destacados de los últimos años. Son también inspiración directa para la implementación de nuestros experimentos, con el objetivo de descubrir cuales de estas técnicas son verdaderamente aplicables a menor escala, en desarrollo indie con recursos limitados en cuanto a hardware y especialización físico-matemática.

2.3.2.1. Towards Generalized Position-Based Dynamics

- Partiendo de sus nuevas ecuaciones para el calculo físico: citechaudhary2025towards we iterate over each individual force term in the system (for example, the force due to one element on its adjacent vertices in a FEM mesh), and update it implicitly using a low-dimensional Newton solve on the element strain. Our proposed scheme is expressive enough to apply to any nonlinear energy-based elastic model

is extremely popular in interactive applications due to its stability, performance, and ease of implementation. Unlike most other iterative techniques used for dynamics simulation, PBD is a so-called dual descent method [Macklin et al. 2020]: it iterates over the forces in the system, and updates the effect of each force one at a time, rather than updating vertex positions as in primal methods. All forces must be expressed as constraints and computed through Lagrange multipliers

d_i is the displacement caused by each force. position-based dynamics can be viewed as a way to iteratively update d_i for each force, and thus obtain the solution x

- Esto permite implementar: Hablan de data driven simulations pero son configuraciones pre grabadas POSES podemos comentarlo como machine learning (en tmbn lo q hace cahos d unreal) En hood aparecen otros paper q tmbn hablan d esto: Learned Deformation Models [20, 30, 39, 42] [20] Peng Guan, Loretta Reiss, David A Hirshberg, Alexander Weiss, and Michael J Black. Drape: Dressing any person. ACM Transactions on Graphics (ToG) [30] Qianli Ma, Jinlong Yang, Anurag Ranjan, Sergi Pujades, Gerard Pons-Moll, Siyu Tang, and Michael J Black. Learning to dress 3d people in generative clothing. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 6469-6478, 2020 [39] Chaitanya Patel, Zhouyingcheng Liao, and Gerard Pons Moll. Tailornet: Predicting

clothing in 3d as a function of human pose, shape and garment style. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 7365-7375, 2020. [42] Igor Santesteban, Miguel A Otaduy, and Dan Casas. Learning-based animation of clothing for virtual try-on. Computer Graphics Forum, 38(2):355-366, 2019

ANTERIORMENTE tambien hemos mencionado CAHOS UNREAL q hace lo mismo....

2.3.2.2. Neural Cloth Simulation

Neural Cloth Simulation [Bertiche et al. \(2022\)](#) propone una metodología no supervisada, unificando simulación y entrenamiento. Sigue el esquema clásico de este tipo de solvers, donde el estado de la tela depende de los instantes previos, gestionado por window frames cuyo tamaño y complejidad dependen del tipo de tela procesada. Su arquitectura consiste en un encoder-decoder recurrente que separa componentes estáticos y dinámicos, haciendo uso de modelos GRU en estos últimos y la cual permite mejorar la generalización y estabilidad del entrenamiento. Este se realiza mediante funciones de pérdida inspiradas en simulación física, incluyendo restricciones físicas en relación a deformaciones, colisiones e inercia. El dataset utilizado es AMASS [Mahmood et al. \(2019\)](#), que recopila información de mas de 40.000 poses, y el modelo físico extiende la idea del modelo mass-spring [Provot et al. \(1995\)](#). En comparación con PBNS o SNUG, esta metodología consigue soluciones más robustas y simples que dan a simulaciones detalladas a costa de mayores tiempo de entrenamiento que sus predecesores, compensando, no obstante, la disminución de recursos al no realizar procesos de generación de datos. Esta perspectiva va de la mano con tendencias actuales en la industria, como la ya mencionada e implementada Chaos Cloth de Unreal Engine.

2.3.2.3. Subspace Neural Physics: Fast Data-Driven Interactive Simulation

Este desasallo de la unidad de investigación Ubisoft La Forge implementa la primera técnica que incorpora reducción de modelo o el uso de subespacios con colisiones con objetos externos en la tela. El estudio se centra en conseguir un balance entre el rendimiento en tiempo real y el coste y memoria necesarios para realizar el precómputo.

El entrenamiento se realiza sobre las posiciones de una simulación realizada en Maya aplanados en un vector, al que se añade otro vector plano con los parámetros necesarios como posición del suelo, valor del viento, o collisionadores con la tela. Se aplica un PCA sobre ambos denotando cuántos ejes de deformación de la tela se tienen en cuenta para representar la complejidad de la recreación: 64, 128, 256; siendo este último el que conserva casi el mismo nivel de detalle que la simulación original.

El modelo que predice la posición del vértice tiene en cuenta los dos frames

anteriores y los colisionadores de ese frame:

$$z_t = \alpha \odot z_{t-1} + \beta \odot (z_{t-1} - z_{t-2})$$

La versión extendida que tiene en cuenta colisiones externas:

$$z_t = \bar{z}_t + \bar{z}_t \odot z_{t-1} w_t^T$$

Aquí dice una cosa importante sobre el error que hay que corregir usando window frames: *We repeat this training procedure on mini-batches of size 16, using overlapping windows of size 32, for around 100 epochs or until training converges. We use a learning rate of 0.0001, and a learning rate decay ratio of 0.999. We use a noise standard deviation σ of 0.01, which we found by visualizing the result of this perturbation in the first 3 components of the PCA space. Training takes between 10 and 48 hours depending on the complexity of the setup and the number of PCA bases used.*

El resultado se carga en una aplicación interactiva 3D propia, dónde se deja al usuario manipular a un personaje con un gamepad. Los cálculos necesarios se aceleran con shaders en GPU (GPU decompression) y se recalculan las normales de los vértices necesarias para el renderizado.

Sus principales limitaciones residen en la necesidad de quitar a mano errores producidos en la simulación, falta de pruebas en objetos plásticos, posibles colisiones internas fallidas, y limitación de objeto colisionador fijo en el espacio y sin contar con objetos cayendo por el espacio. Se necesitaron 10^6 frames de simulación para entrenar a mejor.

2.3.2.4. Cloth and Skin Deformation with a Triangle Mesh Based Convolutional Neural Network

Este estudio es un ejemplo de uso de CNN para la simulación de cloth. Reseña la dificultad de integrar una CNN debido a la construcción de la malla, incompatible antes de su modificación con una grid 2D. Resuelve el problema entrenando mallas "manifold" triangulares: es decir, mallas cuyos bordes están conectados por al menos dos caras (menos los extremos) dejando una separación entre exterior e interior. Mapear la mesh a una imagen 2D es lo que permite realizar la convolución.

Crean una red encoder-decoder formada por bloques UpConv and DownConv con operadores nuevos de convolución, pooling y unpooling. Destacamos como reduce la dimensionalidad de la malla en el pooling: colapasa los edges siguiendo un método en espiral. También señala la importancia de usar Leaky-CNN: para no dejar neuronas muertas, que resulta un problema en la simulación. Además de usar L2 con la L1 para que no se quede el error acumulado en ciertos vértices. También usa

el Adam optimizer.

Este estudio afirma conseguir resultados muy cercanos al *ground truth*, usa base de animaciones (Berkley animation base), cuenta con PC potente, y no obstante, resaltan una vez más los largos tiempos de entrenamiento (entre 4 y 20 horas) y la necesidad de calidad y cantidad de datos de entrenamiento. Tampoco asegura deformaciones válidas en cualquier caso.

Este estudio, a diferencia del anterior, incluye brazos y manos (joints). Finalmente

+ Cloth unsampling, Pose to cloth, PCA back to original space.

2.3.3. DeepSD: Real-time Cloth Simulation with Deep Learning

CNN. Tambien esta deep wrinkles

2.3.3.1. PBNS: Physically Based Neural Simulation for Unsupervised Garment Pose Space Deformation

PNBS [Bertiche et al. \(2021\)](#) presenta una simulación neuronal DL no supervisada, como alternativa al enfoque clásico PBS. Integra principios físicos en el entrenamiento con funciones físicas de energía, entre ellas restricciones de fijación. En la arquitectura encontramos un perceptron multicapa (MLP) con funciones ReLU que genera deformaciones tipo PSD sobre modelos LBS, cuya aplicación en el mundo 3D garantiza la compatibilidad con los motores gráficos y su alta eficiencia. Esta metodología entrena sobre datasets públicos como AMASS [Mahmood et al. \(2019\)](#) y CMU MoCap, siendo capaz de manejar colisiones en poses extremas, así como colisiones cloth-to-cloth. Asimismo, permite la obtención de prendas que se ajustan a diferentes cuerpos (garment resizing). En comparación a otros métodos como TailorNet [Patel et al. \(2020\)](#), logra una mayor consistencia física y generalización, además de un coste computacional considerablemente menor, incluso sin GPU. Sin embargo, se ve limitado por su enfoque en poses estáticas.

2.3.3.2. SNUG

[citesantesteban2022snug](#) Sistema de muelles tmbn (fuerza bruta) pero entrenado en una red recurrente para conseguir equilibrio estático (ley física) lo que hace que pueda ser preentrenado NO supervisado y NO necesitar base de datos de mallas de CLOTH ya simuladas Las bases d datos en las que cuenta son los modelos en movimiento (underlying body parameters and motion) AMASS dataset [33], totaling 6,519 frames

LAS NORMAS FISICAS DE LAS QUE HABLAMOS PARA EL SELF SUPERVISION: - Membrane Strain Energy: Energia transmitida entre dos puntos en base a la elasticidad del material. - Bending Energy: El ángulo diédrico entre dos caras en relación a su propiedad de rigidez. - Gravity: Gravedad. - Collision Penalty: Distancia prudencial al modelo del cuerpo.

2.3.3.3. HOOD: Hierarchical Graphs for Generalized Modelling of Clothing Dynamics

2023 avance!!!! tres pasos importantes

a) graph neural networks that learn the local dynamics in a garment-independent way: trains its network using physics instead of labeled examples. el grafo sirve para propagar mensajes (pura física) NO SÉ SI PLANTEAMOS LLEGAR TAN LEJOS LOL: Our method learns to predict physically realistic fabric behavior by reasoning about the map between local deformations, forces, and accelerations. Thanks to its locality, our method is agnostic to both the global structure and shape of the garment and directly generalizes to arbitrary body shapes and motions.

b) hierarchical message-passing for the efficient capture of long-range coupling: related a MeshGraphNets pero con más mensajes (multi-level) y entre varias estrategias de pooling que se están llevando ellos hacen: graph coarsening strategy that allows our network to implicitly learn transitions between graph levels

c) a physics-based loss function that enables self-supervised training: utiliza incremental potential for implicit time stepping como loss function en esto se refiere a la propia ecuación de simulación física: implicit time integration, where the next state of the cloth is found by minimizing an energy (potential) that encodes: stretching / bending energies, inertia (how velocity continues), contact, friction, etc. This energy (often called an incremental potential) is a scalar function of the cloth configuration at the next time step. Aplicando la propia física correcta como loss, consigue aprendizaje NO SUPERVISADO

Soluciona PROBLEMAS q otros estudios: - huesos a la tela para poder extender de las prendas apretadas - telas como point clouds para poder adaptarse a la topología lo coloca dentro d Learned Deformation Models

Limites: Velocidad d sim, self-coll...

Capítulo 3

Descripción del Trabajo

3.1. SimulacionUnity

3.2. Dataset

UVs utilizadas para aproximarnos a graph nns pero d manera barata para nosotros

3.3. EntrenamientoPython

se acumula el error: - segundo modelo // segunda comprobación - red recurrente
-

3.4. SimulacionUnity

3.5. ModelosFuncionales

3.6. PruebaTesters

Conclusiones y Trabajo Futuro

4.1. Conclusiones

Las conclusiones de cada una de las partes del trabajo se presentan en las siguientes secciones de manera más detalladas.

4.1.1. Conclusiones de la búsqueda de datasets

Bla.

4.1.2. Conclusiones de la blablabla

Bla.

4.1.3. Limitaciones

Este estudio ha tenido una serie de limitaciones, desde blablabla Los modelos de redes neuronales no han dado los resultados esperados, esto puede ser por la falta de datos, que aunque haya un número que a simple vista parece suficiente no lo es para entrenar redes neuronales, por las limitaciones de hardware para entrenar modelos más complejos y por los tiempos requeridos para la realización de este estudio no ser lo suficientemente largos como para llevar a cabo entrenamientos más extensos y con mayor búsqueda de hiperparámetros.

Todas estas limitaciones, han llevado al siguiente plan de trabajo a futuro.

4.2. Trabajo Futuro

El trabajo a futuro de este estudio se puede centrar en varios esfuerzos:

- bla
- bla

- bla

Contribuciones Personales

Eva

Muchas cosas.

Pau

Muchas cosas.

Mik

Muchas cosas.

Bibliografía

The sciences, each straining in its own direction, have hitherto harmed us little; but some day the piecing together of dissociated knowledge will open up such terrifying vistas of reality, and of our frightful position therein, that we shall either go mad from the revelation or flee from the deadly light into the peace and safety of a new dark age.

H.P. Lovecraft, *The Call of Cthulhu*

BERTICHE, H., MADADI, M. y ESCALERA, S. Pbns: Physically based neural simulation for unsupervised garment pose space deformation. *ACM Transactions on Graphics (TOG)*, vol. 40(6), páginas 1–14, 2021.

BERTICHE, H., MADADI, M. y ESCALERA, S. Neural cloth simulation. *ACM Transactions on Graphics (TOG)*, vol. 41(6), páginas 1–14, 2022.

CONTRIBUTORS, P. Pytorch documentation. <https://docs.pytorch.org/docs/stable/index.html>, 2026. Accessed: April 17, 2026.

GAMES, E. Chaos cloth. <https://dev.epicgames.com/documentation/unreal-engine/machine-learning-cloth-simulation-overview>, 2025. Accessed: (17/04/2026).

HAVOK. Havok physics. <https://www.havok.com/havok-physics/>, 2026. Accessed: 2026-04-28.

HECKER, C. Physics in computer games (title only). *Communications of the ACM*, vol. 43(7), páginas 34–39, 2000.

JIM HUGUNIN, U. Unite 2016 - gpu accelerated high resolution cloth simulation. <https://www.youtube.com/watch?v=kCGHX1LR3l8>, 2016. Accessed: (15/04/2026).

KENNY ERLEBEN, K. H., JON SPORRING. *Physics-based Animation*. Charles River Media, 2005.

- MACKLIN, M., MÜLLER, M. y CHENTANEZ, N. XPBD: position-based simulation of compliant constrained dynamics. En *Proceedings of the 9th International Conference on Motion in Games*, páginas 49–54. ACM, Burlingame California, 2016. ISBN 978-1-4503-4592-7.
- MAHMOOD, N., GHORBANI, N., TROJE, N. F., PONS-MOLL, G. y BLACK, M. J. Amass: Archive of motion capture as surface shapes. páginas 5442–5451, 2019.
- METHOD, V. Obi cloth. <https://assetstore.unity.com/packages/tools/physics/obi-cloth-81333?aid=1011134eQ>, 2019. Accessed: (16/04/2026).
- MILLINGTON, I. *Game Physics Engine Development*. CRC Press, 2007. ISBN 9781482267327.
- MÜLLER, M., HEIDELBERGER, B., HENNIX, M. y RATCLIFF, J. Position based dynamics. *Journal of Visual Communication and Image Representation*, vol. 18(2), páginas 109–118, 2007. ISSN 10473203.
- NVIDIA. Physx sdk. <https://developer.nvidia.com/physx-sdk>, 2026. Accessed: 2026-04-9.
- PATEL, C., LIAO, Z. y PONS-MOLL, G. Tailornet: Predicting clothing in 3d as a function of human pose, shape and garment style. En *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, páginas 7365–7375. 2020.
- PROJECT, O. Onnx: Open neural network exchange. <https://onnx.ai/>, 2026. Accessed: 2026-04-17.
- PROVOT, X. Deformation Constraints in a Mass Spring Model to Describe Rigid Cloth Behavior. 1995.
- PROVOT, X. ET AL. Deformation constraints in a mass-spring model to describe rigid cloth behaviour. En *Graphics interface*, páginas 147–147. Canadian Information Processing Society, 1995.
- SALEN, K. y ZIMMERMAN, E. *Rules of Play: Game Design Fundamentals*. MIT Press, 2004.
- TECHNOLOGIES, U. Physics integrations. <https://docs.unity3d.com/Manual/physics-integrations.html>, 2026a. Accessed: 2026-04-28.
- TECHNOLOGIES, U. Unity sentis: Ai inference package v2.4. <https://docs.unity3d.com/Packages/com.unity.ai.inference@2.4/manual/index.html>, 2026b. Accessed: 2026-04-17.
- TEMPLET, R. M. *Game Physics: An Analysis of Physics Engines for First-Time Physics Developers*. Tesis Doctoral, California State University, Northridge, 2021.
- TOWER, S. The big game engines report. https://app.sensortower.com/vgi/assets/reports/The_Big_Game_Engines_Report_of_2025.pdf, 2025. Accessed: 2026-04-28.

-
- VA, H., CHOI, M.-H. y HONG, M. Real-time cloth simulation using compute shader in unity3d for ar/vr contents. *Applied Sciences*, vol. 11(17), 2021. ISSN 2076-3417.

Carteles para la generación del dataset



Figura A.1: Cartel colgado en la facultad de informática

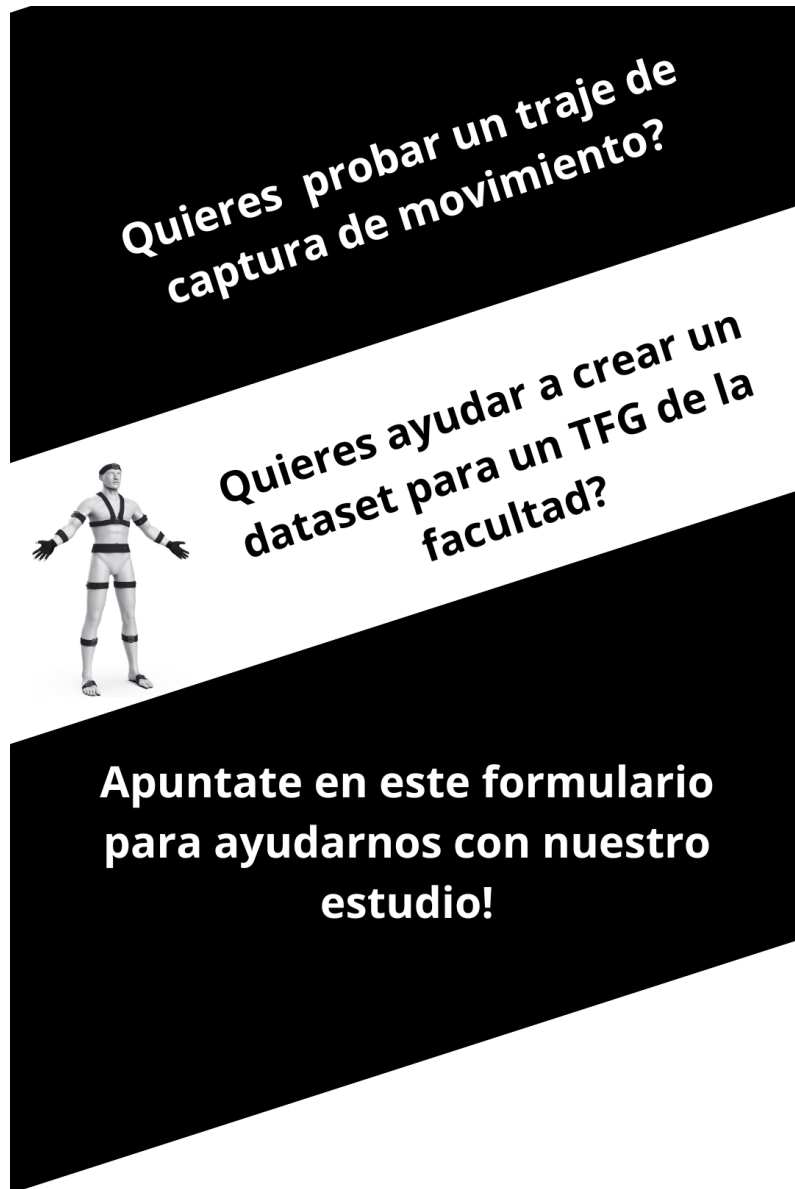


Figura A.2: Cartel colgado en redes sociales



Figura A.3: Cartel colgado en pantallas de la facultad

Formulario de recogida de citas

Participación Dataset para detección de poses

Necesitamos datos para un TFG de la Facultad de Informática de la UCM. Lo importante, **¿qué datos recopilamos?** La siguiente lista de datos son los datos que vamos a recopilar y hacer públicos en el propio estudio del tfg o en un dataset público en [kaggle](#):

- Edad
- Sexo
- Nacionalidad
- Datos de coordenadas de distintas partes del cuerpo dadas por un traje de captura de movimiento

Estos datos se recogerán en un formulario a parte el día de la prueba, siendo hechos desde un correo de los organizadores y permaneciendo totalmente anónimos sin correlación a la persona que ha realizado la prueba.

OBJETIVO DE LOS DATOS: los datos de edad, nacionalidad y sexo son meramente estadísticos, solo se usarán para demostrar la heterogeneidad (o falta de) de los datos tomados. Los datos de coordenadas se usarán para entrenar distintos modelos de clasificación de pose con la intención de poder decir con acierto el gesto o pose que está haciendo una persona mientras lleva el traje puesto.

LOCALIZACIÓN DE LA PRUEBA: [Facultad de Informática \(UCM\)](#), Calle del Prof. José García Santesmases, 9, Moncloa - Aravaca, 28040 Madrid

alejba02@ucm.es

Cambiar de cuenta

* Indica que la pregunta es obligatoria

Correo *

Tu dirección de correo electrónico

¿Aceptas la recogida de datos y la participación en el muestreo? *

Si

No

Siguiente

Página 1 de 2

Borrar formulario

Este formulario se creó en Universidad Complutense de Madrid.

¿Parece sospechoso este formulario? [Informe](#)

Figura B.1: Formulario de recogida de citas (primera parte)

29

Participación Dataset para detección de poses

alejba02@ucm.es [Cambiar de cuenta](#)

Horario disponible

A continuación marca las horas en las podrías estar disponible. Con la respuesta que proporciones te mandaremos un correo con un día y hora determinados.

Lunes

- ☐ 9:00-10:00
- ☐ 10:00-11:00
- ☐ 11:00-12:00
- ☐ 12:00-13:00
- ☐ 13:00-14:00
- ☐ 14:00-15:00
- ☐ 15:00-16:00
- ☐ 16:00-17:00
- ☐ 17:00-18:00
- ☐ 18:00-19:00
- ☐ 19:00-20:00

Figura B.2: Formulario de recogida de citas (segunda parte)

Resultados de los distintos modelos CNN

La línea verde en los gráficos de tensorboard indican la mejor combinación de hiperparámetros encontrada en cada caso

C.1. Intervalo 0.2s

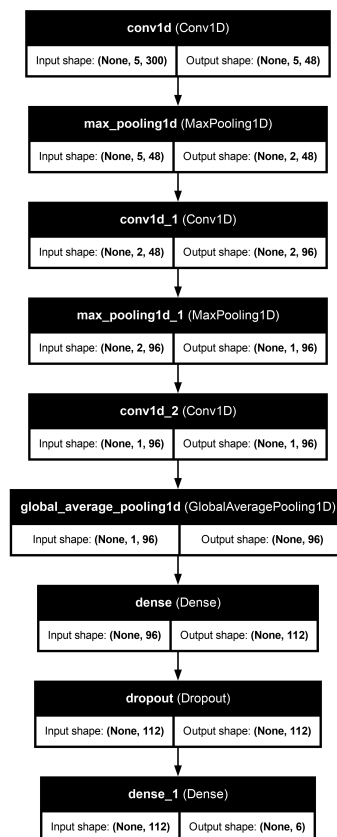


Figura C.1: Esquema del modelo CNN con 0.2s de intervalo

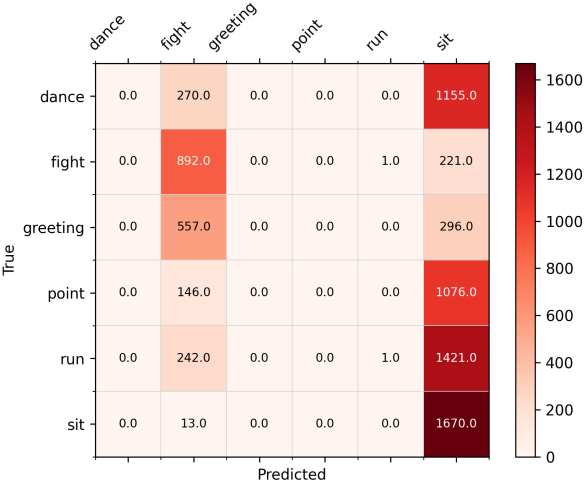


Figura C.2: Matriz de confusión del modelo CNN con 0.2s de intervalo

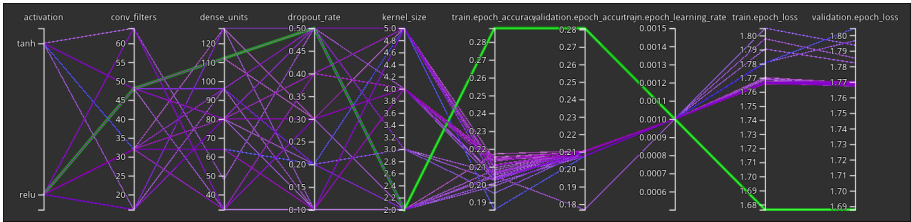


Figura C.3: Gráfico de entrenamiento del modelo CNN con 0.2s de intervalo (mejor val_accuracy = 0.28777)

C.2. Intervalo 0.4s

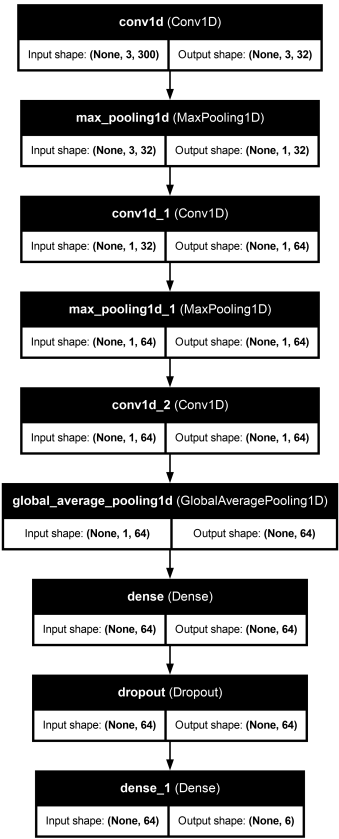


Figura C.4: Esquema del modelo CNN con 0.4s de intervalo

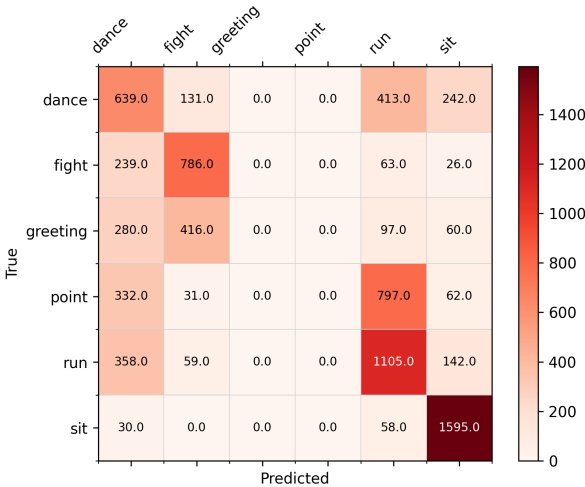


Figura C.5: Matriz de confusión del modelo CNN con 0.4s de intervalo

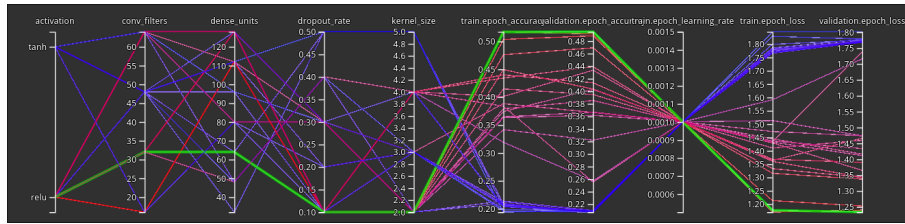


Figura C.6: Gráfico de entrenamiento del modelo CNN con 0.4s de intervalo (mejor $\text{val_accuracy} = 0.49473$)

C.3. Intervalo 0.6s

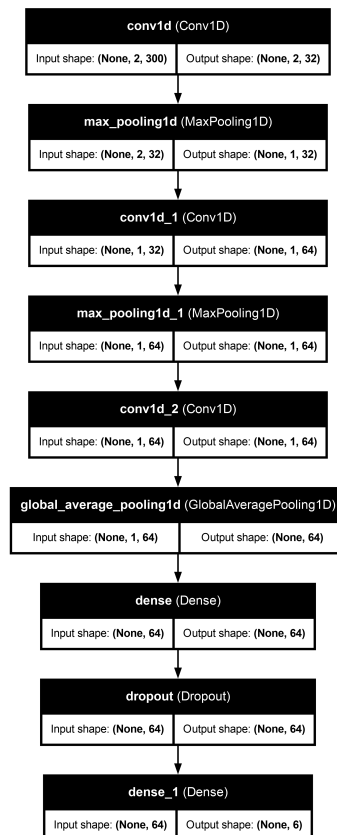


Figura C.7: Esquema del modelo CNN con 0.6s de intervalo

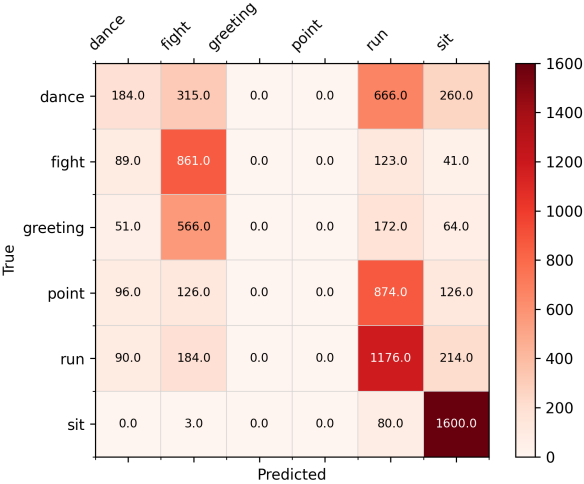


Figura C.8: Matriz de confusión del modelo CNN con 0.6s de intervalo

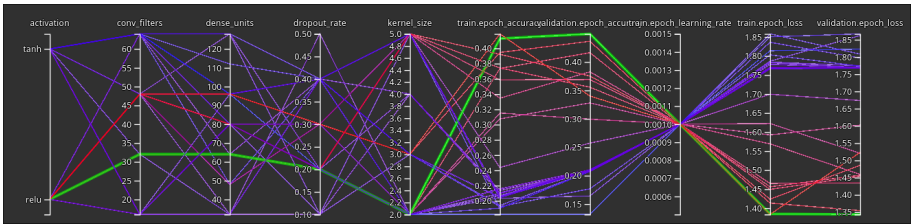


Figura C.9: Gráfico de entrenamiento del modelo CNN con 0.6s de intervalo (mejor val_accuracy = 0.44952)

C.4. Intervalo 0.8s

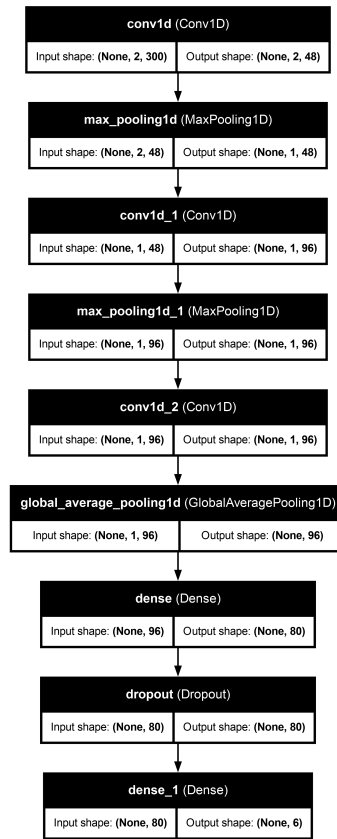


Figura C.10: Esquema del modelo CNN con 0.8s de intervalo

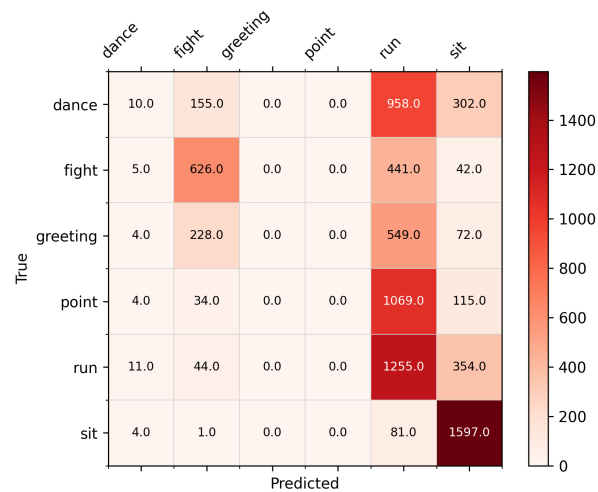


Figura C.11: Matriz de confusión del modelo CNN con 0.8s de intervalo

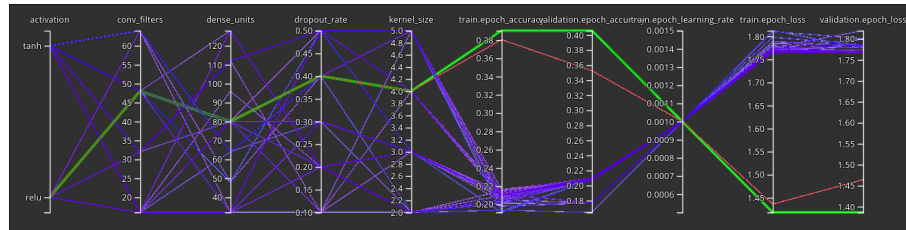


Figura C.12: Gráfico de entrenamiento del modelo CNN con 0.8s de intervalo (mejor val_accuracy = 0.40583)

C.5. Intervalo 1.0s

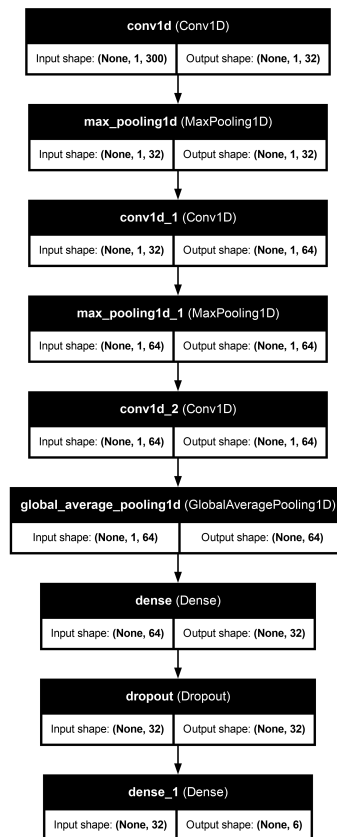


Figura C.13: Esquema del modelo CNN con 1.0s de intervalo



Figura C.14: Matriz de confusión del modelo CNN con 1.0s de intervalo

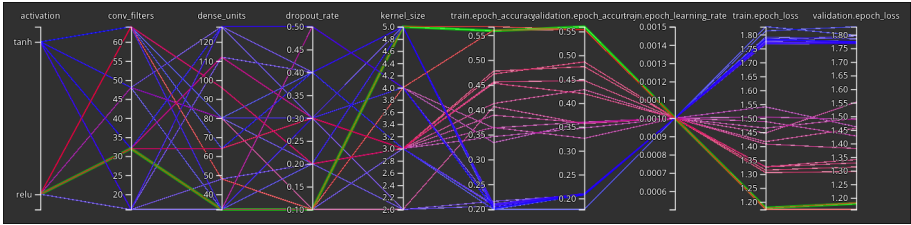


Figura C.15: Gráfico de entrenamiento del modelo CNN con 1.0s de intervalo (mejor val_accuracy = 0.56002)

Glosario

CNN Convolutional Neural Network

Licencia de uso del documento

Esta obra está bajo una licencia [Creative Commons](http://creativecommons.org/licenses/by-sa/4.0/legalcode) «Atribución-NoComercial-CompartirIgual 4.0 Internacional».



<http://creativecommons.org/licenses/by-sa/4.0/legalcode>.

Licencia de uso del código fuente

Los archivos de código fuente para generar este documento se encuentran en <https://github.com/FratosVR/Memoria-TFG> bajo la licencia [GPL-3.0](#)